Python for Data Science

Week(3) : Core Data Structures



GitHub Repository



Moodle

Dr. Ghadeer Mobasher

Agenda

1. **Lists and indexing principles**
   Lists and corresponding operations

2. **Tuples and immutability**
   Tuples vs Lists

3. **Sets and Uniqueness**
   Sets and corresponding operations

4. **Dictionaries**
   Key-value modelling

5. **Nested Data Structures**
   Real-world Data Modelling

6. **Putting it all together**
   Final Mini Challenge: Course Registry Analyser

Lists

Definition:

Lists are ordered, mutable sequences that allow storing multiple items in a single variable.

```
numbers = [10, 20, 30, 40]
```

Key Features:

- Ordered (elements maintain insertion order)
- Mutable (elements can be added, removed, or modified)
- Indexed (zero-based indexing, supports negative indices and slicing)

```
# empty list
empty_list = list()
print(len(empty_list))
```



```
0
```

Append and Insert Operations

```
# Append
fruits = ['cherry', 'apple', 'pineapple', 'mango']
fruits.append('coconut')
print(fruits)
fruits.append('lime')
print(fruits)
```

```
['cherry', 'apple', 'pineapple', 'mango', 'coconut']
['cherry', 'apple', 'pineapple', 'mango', 'coconut', 'lime']
```

```
# insert
fruits = ['cherry', 'apple', 'pineapple', 'mango']
fruits.insert(2, 'coconut')
print(fruits)
fruits.insert(3, 'lime')
print(fruits)
```

```
['cherry', 'apple', 'coconut', 'pineapple', 'mango']
['cherry', 'apple', 'coconut', 'lime', 'pineapple', 'mango']
```

List Comprehension

```
#  List comprehensions to reverse a list
fruits = ['cherry', 'apple', 'pineapple', 'mango']
reserved_fruits= [fruit for fruit in fruits [::-1]]
print(reserved_fruits)
```

```
['mango', 'pineapple', 'apple', 'cherry']
```

```
# List comprehensions to power and reverse number
numbers = [1, 2, 3, 4, 5]
squared_reversed = [num**2 for num in numbers[::-1]]
print(squared_reversed)
```

```
[25, 16, 9, 4, 1]
```

In **Week3.ipynb** , check the beginning of the notebook until the end of section **“1. Lists”**

In **Week3_Exercise.ipynb** , you have 20 minutes to solve “Section A: Lists”.

First, download this Colab Notebook and work via VS or Jupyter Notebook

Tuples

Definition: Tuples are ordered, immutable sequences.

Code Example

```
point = (3, 5)
```

Key Features:

- Ordered
- Immutable (cannot modify elements)
- Hashable (can be used as dictionary keys if elements are hashable)

Use cases: Fixed records, coordinates, and multiple return values from functions.

```
tuple1 = ('item1', 'item2', 'item3')
first_item = tuple1[0]
print(first_item)
second_item = tuple1[1]
print(second_item)
```

```
item1
item2
```

```
fruits = ('apple', 'cherry', 'mango', 'lemon')
fruits[0]='orange' #TypeError: 'tuple' object does not support item assignment
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[35], line 2
      1 fruits = ('apple', 'cherry', 'mango', 'lemon')
----> 2 fruits[0]='orange'

TypeError: 'tuple' object does not support item assignment
```

Sets

Definition: A set is an unordered collection of unique elements.

Code Example

```
numbers = {1, 2, 3, 3, 4}
```

Key Features:

- No duplicates
- Unordered
- Efficient membership testing

Use cases:

- Unique identifiers
- Categories
- Membership relations

```
# membership
item_set = {'item1', 'item2', 'item3', 'item4'}
print("Does item_set contain item3? ", 'item3' in item_set)
```

```
Does item_set contain item3?  True
```

```
# update - Insert the items from set y into set x
item_set = {'item1', 'item2', 'item3', 'item4'}
item_set.update({'item5','item6','item7'})
print(item_set)
```

```
{'item6', 'item4', 'item5', 'item1', 'item7', 'item3', 'item2'}
```

In **Week3.ipynb** , check sections (“2. Tuples and Immutability“ and “3. Sets“)

In **Week3_Exercise.ipynb** , you have 30 minutes to solve “Section B: Tuples and Sets” .

Pause / Break

See you in 15 minutes ☺!
Please be on time..

Dictionaries

Definition: A dictionary is a mapping from keys to values (**key** → **value**)

Code Example

```
student = {
    "name": "Anna",
    "age": 23,
    "programme": "Computer Science"
```

Key Features:

- Keys must be immutable
- Fast lookup

Use cases:

- Configuration settings
- Structured records
- JSON-like data modelling

```
keys_values = {
    'key1':'value1',
    'key2':'value2',
    'key3':'value3',
    'key4':'value4'}
print(keys_values.keys())
print(keys_values.values())
```



```
dict_keys(['key1', 'key2', 'key3', 'key4'])
dict_values(['value1', 'value2', 'value3', 'value4'])
```

```
# update ()
keys_values = {
    'key1':'value1',
    'key2':'value2',
    'key3':'value3',
    'key4':'value4'}
updated_key3= {'key3':'value34'}
keys_values.update(updated_key3)
updated_key1= {'key1':'value3'}
keys_values.update(updated_key1)
print(keys_values)
```



```
{'key1': 'value3', 'key2': 'value2', 'key3': 'value34', 'key4': 'value4'}
```

Creating a list of tuples from two lists using zip()

```
# define a list of keys
keys = ['a', 'b', 'c', 'd', 'e']

# define a corresponding list of values
values = [1, 2, 3, 4, 5]

# zip(keys, values) pairs each key with its corresponding value as tuples:
# ('a', 1), ('b', 2), ('c', 3), ...
result = zip(keys, values)
# returns an iterator object, not a list. It generates the pairs as you iterate over it.
print (result)
# list() function consumes the iterator and displays all its pairs as a list of tuples
print(list(result))
```



```
<zip object at 0x00000213ADA9E1C0>
[('a', 1), ('b', 2), ('c', 3), ('d', 4), ('e', 5)]
```

Dictionary Comprehension

```
# define a list of keys
keys = ['a', 'b', 'c', 'd', 'e']

# define a corresponding list of values
values = [1, 2, 3, 4, 5]
```



```
{'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
```

Dictionary Comprehension (Cont.)

```
# creating a dictionary from two lists using zip()

# define a list of keys
keys = ['a', 'b', 'c', 'd', 'e']

# define a corresponding list of values
values = [1, 2, 3, 4, 5]

# zip(keys, values) pairs each key with its corresponding value as tuples:
# ('a', 1), ('b', 2), ('c', 3), ...
# the result is an iterator, not a list

# dictionary comprehension:
# iterate over each (key, value) pair produced by zip
# construct a dictionary where each key is mapped to its corresponding value
dict = {k: v for (k, v) in zip(keys, values)}

# print the resulting dictionary
print(dict)
```

Assignment target → resulting dictionary

Key expression

Value expression

Receives each tuple and unpacks it into k and v

Calls zip() and returns an iterator that supplies pairs.

```
{'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
```

Dictionary Comprehension (Cont.)

```
# definition by comprehension
squared_dict = {x: x**2 for x in range(1, 6)}
print(squared_dict)
```



```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```



In **Week3.ipynb** , check sections (“4. Dictionaries and Key-Value Modelling“)



In **Week3_Exercise.ipynb** , you have 30 minutes to solve “Section C: Dictionaries” .

```
# Dictionary Comprehension with Conditional Statements
dict = {x: x**2 for x in range(10) if x**2 % 2 == 0}
print(dict)
```



```
{0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

Nested Data Structure

Definition: Combinations of lists, dictionaries, tuples, and sets to model complex, real-world data structures.

Code Example: List of dictionaries

```
students = [
    {"name": "Anna", "grade": 1.7},
    {"name": "Ben", "grade": 2.3},
    {"name": "Clara", "grade": 1.3}
]
students[0]["name"]   # "Anna"
```

```
course = {
    "title": "Data Science",
    "participants": ["Anna", "Ben", "Clara"]
}

course["participants"][1]  # "Ben"
```

Use cases: Student registries, banking systems, research datasets, JSON-like data modeling.

Nested Dictionary & Data Structure

```
actor = {
'first_name':'Tom',
'last_name': 'Hanks',
'country': 'United States',
'city': 'Concord, California',
'age': 69,
'is_married':False,
'skills':['Comedy Actor', 'Drama Actor', 'Filmmaker'],
'person_info' : { # Nested
    'firstname': 'Tom',
    'lastname': 'Hanks',
    'country': 'United States',
    'city': 'Concord, California',
'children': [' Colin Hanks','Elizabeth Hanks','Chet Hanks','Truman Hanks']

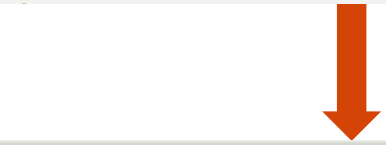
    }
}
```

```
print(actor['first_name'])
print(actor['last_name'])
print(actor['country'])
print(actor['city'])
print(actor['age'])
print(actor['is_married'])
print(actor['skills'])
print(actor['skills'][0])
print(actor['person_info']['children'][1:4])
```

```
Tom
Hanks
United States
Concord, California
69
False
['Comedy Actor', 'Drama Actor', 'Filmmaker']
Comedy Actor
['Elizabeth Hanks', 'Chet Hanks', 'Truman Hanks']
```

Flattening a Matrix using Comprehension

```
# Flattening a List using comprehension
'''matrix is defined as a 2D list (a list of lists) containing 4 rows of 4 integers each''
matrix = [
    [9, 3, 8, 3],
    [4, 5, 2, 8],
    [6, 4, 3, 1],
    [1, 0, 4, 5],
]
```



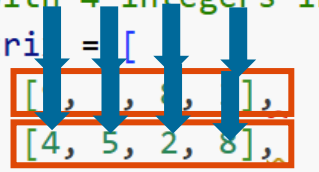
```
[9, 3, 8, 3, 4, 5, 2, 8, 6, 4, 3, 1, 1, 0, 4, 5]
```

Flattening a Matrix using Comprehension (Cont.)

```
# Flattening a 2D list using list comprehension

# Define a 2D list (a list of lists) containing 4 rows
# with 4 integers in each row.
matrix = [
    [9, 3, 8, 3],
    [4, 5, 2, 8],
    [6, 4, 3, 1],
    [1, 0, 4, 5],
]

# List comprehension that:
# - Iterates through each row in the matrix
# - Iterates through each item in the current row
# - Collects every item into a new, flat 1D list.
print([item for row in matrix for item in row])
```



for each `row` in `matrix`, iterate over each `item` in that `row`, and collect `item`.



```
[9, 3, 8, 3, 4, 5, 2, 8, 6, 4, 3, 1, 1, 0, 4, 5]
```

A precise explanation is:

- `matrix` is the **outer iterable**
- `row` is each element in `matrix` (each inner list)
- `row` then becomes a new iterable
- `item` is each element inside that `row`



In **Week3.ipynb** , check sections (**“5. Nested Data Structures“**)



In **Week3_Exercise.ipynb** , you have 35 minutes to solve **“Section D: Nested Data Structures ”** .

Final Mini Challenge: Course Registry Analyser

Final Mini Challenge: Course Registry Analyser

You are given the enrolment records of a university course registry. Each student has a name and a dictionary of the courses they take, together with their score in each course.

Given Data

```
registry = [
    {"name": "Ali",  "scores": {"Python": 85, "Statistics": 72, "NLP": 90}},
    {"name": "Sara", "scores": {"Python": 58, "Statistics": 64}},
    {"name": "Lina", "scores": {"Python": 73, "Statistics": 69, "NLP": 81}},
    {"name": "Omar", "scores": {"Statistics": 49, "NLP": 55}},
    {"name": "Noor", "scores": {"Python": 91, "Statistics": 88, "NLP": 94}},
    {"name": "Yara", "scores": {"Python": 66, "NLP": 60}}
]
```

Tasks

Part 1: Course overview



In **Week3_Exercise.ipynb** , you have 35 minutes to solve the “Final Mini Challenge”

What did we learn today?

Lists and indexing Principles

Lists and corresponding operations

Tuples and Immutability

Tuples vs Lists

Sets and Uniqueness

Sets and corresponding operations

Dictionaries

Key-value Modelling

Nested Data Structures

Real-world Data Modelling

Coming Up : Week (4)

So far, we have worked with core data structures to store and organise related information. How can functions help us reuse this logic and divide a program into smaller, manageable parts?

• Purpose of functions • Parameters and return values
• Variable scope • Default parameters
• Lamda Expressions • Writing reusable and modular code

Any Questions?

See you next time ☺

Dr. Ghadeer Mobasher
ghadeer.mobasher@srh-hochschulen.de
https://www.linkedin.com/in/ghadeermobasher/
LGS 6, 2. OG, Arc 213